

Conceptos básicos

1. Naturaleza PHP

- **¿Qué es PHP?** : PHP (acrónimo recursivo de PHP: Hypertext Preprocessor) es un lenguaje de programación de código abierto especialmente diseñado para el desarrollo web del lado del servidor, cuya principal característica es la capacidad de incrustarse directamente en el código HTML, permitiendo la generación de contenido dinámico antes de que el servidor envíe los datos al navegador.
Se utiliza para gestionar bases de datos, sesiones de usuario, comercio electrónico y es la base de los sistemas de gestión de contenidos (CMS) más utilizados, como WordPress o Drupal, siendo que su evolución lo ha transformado de una simple herramienta de plantillas a un lenguaje robusto con soporte completo para programación orientada a objetos.
- **Lenguaje interpretado** : El código no se compila en un archivo ejecutable binario previo a su distribución, sino que un componente llamado intérprete (generalmente el motor Zend) lee y ejecuta las instrucciones línea por línea en tiempo de ejecución, de esta forma facilitando enormemente el ciclo de desarrollo, ya que los cambios se visualizan inmediatamente al recargar la página. Internamente.
Se utiliza en PHP como un proceso de "compilación al vuelo" donde el código fuente se transforma en opcodes (instrucciones intermedias) que luego son procesadas por la CPU, para optimizar esto en entornos de producción, se suele utilizar OPcache, que almacena estos códigos ya procesados para evitar re-interpretar el script en cada solicitud.
- **Tipado dinámico** : Tipo de dato de una variable que no se declara explícitamente al definirla, sino que se determina automáticamente en tiempo de ejecución según el valor que se le asigne. permitiendo de esta forma que una misma variable pueda cambiar de tipo a lo largo de su ciclo de vida (por ejemplo, pasar de ser un integer a un string sin generar errores).
Se utiliza para agilizar la escritura de código y reducir la verbosidad, permitiendo al desarrollador centrarse en la lógica de negocio en lugar de la gestión estricta de la memoria o la declaración de estructuras.
- **Tipado débil y coerción de tipos** : Capacidad del lenguaje para realizar operaciones entre diferentes tipos de datos sin requerir conversiones manuales explícitas (casting), se logra mediante la coerción de tipos, un mecanismo donde PHP convierte internamente un valor a otro tipo compatible para completar una operación, por ejemplo si se intenta sumar el número 5 (entero) con la cadena "10" (string), PHP transformará automáticamente la cadena en un entero para devolver 15.
Se utiliza para dar flexibilidad y evitar interrupciones por incompatibilidades menores, aunque requiere precaución para evitar resultados inesperados en comparaciones lógicas.
- **strict_types (concepto)** : Directiva introducida en PHP 7 que permite desactivar la coerción de tipos automática en llamadas a funciones y retornos de datos dentro de

un archivo específico, cuando se activa mediante `declare(strict_types=1)`, el intérprete lanzará un error fatal (TypeError) si el tipo de dato pasado no coincide exactamente con el tipo definido en la firma de la función.

Se utiliza en el desarrollo profesional para garantizar la integridad de los datos, reducir errores lógicos difíciles de depurar y forzar al desarrollador a escribir un código más predecible y seguro, similar al comportamiento de lenguajes de tipado fuerte como Java o C#.

- **Ciclo request–response** : Modelo fundamental de funcionamiento de PHP en la web en el que el ciclo comienza cuando un cliente (navegador) envía una Request (petición HTTP) al servidor web (como Apache o Nginx), el servidor identifica que el archivo solicitado es un script PHP y lo deriva al intérprete, luego PHP procesa la lógica, consulta bases de datos o archivos, y genera una salida (generalmente HTML, JSON o XML) y finalmente el servidor envía esa salida de vuelta al navegador como una Response (respuesta HTTP) y el script finaliza su ejecución, liberando la memoria.

PHP es "sin estado" (stateless), lo que significa que cada petición es independiente y el lenguaje no "recuerda" la ejecución anterior a menos que se usen sesiones o bases de datos.

2. Motor del lenguaje

- **Zend Engine** : Corazón de PHP, un motor de ejecución de código abierto escrito en C que interpreta y ejecuta los scripts, que actúa como la capa intermedia entre el código fuente escrito por el desarrollador y el hardware del servidor y su función principal es analizar la sintaxis (parsing), gestionar la memoria, manejar los recursos del sistema y proporcionar la infraestructura para las funciones estándar y la programación orientada a objetos.

Se utiliza como el estándar de la industria desde PHP 4, permitiendo que el lenguaje sea multiplataforma y eficiente al abstraer la complejidad del sistema operativo subyacente.

- **Compilación a bytecode** : Aunque PHP es un lenguaje interpretado, no lee el código fuente directamente en cada paso de ejecución, en su lugar realiza un proceso interno de compilación a bytecode, que cuando un script es solicitado, el motor realiza un análisis léxico y sintáctico para transformar el código de alto nivel en una serie de instrucciones de bajo nivel denominadas bytecodes, este paso intermedio es crucial porque permite que el motor valide la estructura del código antes de intentar procesarlo.

Se utiliza para optimizar la velocidad, ya que estas instrucciones son mucho más sencillas de procesar para la máquina virtual que el texto plano original del archivo .php.

- **Ejecución mediante opcodes** : Unidades individuales de instrucción resultantes de la compilación a bytecode que la máquina virtual de Zend entiende y ejecuta, en que cada línea de PHP se traduce en uno o varios opcodes (como ZEND_ADD para una suma o ZEND_ECHO para imprimir texto), el proceso de ejecución consiste en recorrer estas instrucciones de forma secuencial dentro del motor.

Se utiliza para separar la fase de "entendimiento" del código de la fase de "acción",

permitiendo que herramientas como OPcache almacenen estos opcodes en la memoria RAM, evitando así repetir las fases de análisis y compilación en peticiones posteriores, lo que aumenta drásticamente el rendimiento en entornos de producción.

3. Integración con HTML

- **¿Qué es?** : Capacidad nativa de PHP para coexistir dentro de un mismo archivo con etiquetas de marcado web, permitiendo la creación de plantillas dinámicas, a diferencia de otros lenguajes que requieren motores de plantillas externos, PHP actúa como un procesador de texto: el servidor lee el archivo .php, ejecuta únicamente la lógica programada y devuelve al navegador un documento HTML puro.

Se utiliza para inyectar datos provenientes de bases de datos, realizar validaciones de formularios o mostrar contenido condicional (como menús de usuario logueado) sin romper la estructura visual del sitio, facilitando un flujo de trabajo híbrido entre diseño y programación.

- **Etiquetas PHP `<?php ?>`** : Actúan como delimitadores que indican al motor de PHP exactamente dónde comienza y termina el código que debe ser procesado, todo lo que se encuentre fuera de estos delimitadores es ignorado por el intérprete y enviado directamente al flujo de salida como texto plano o HTML.
Se utilizan para "escapar" del modo HTML al modo PHP, además, existe una variante abreviada para impresión directa, "`<?= $variable ?>`", que equivale a "`<?php echo $variable; ?>`", el uso de la etiqueta de cierre es obligatorio cuando el archivo contiene una mezcla de PHP y HTML, pero se recomienda omitirla en archivos que contienen exclusivamente código PHP para evitar el envío accidental de espacios en blanco o cabeceras HTTP prematuras.
- **Incrustación de PHP en HTML** : Técnica de insertar fragmentos lógicos de PHP dentro de la estructura de etiquetas HTML para manipular el DOM (Document Object Model) de forma dinámica antes de que llegue al cliente, se manifiesta comúnmente en el uso de estructuras de control alternativas (como `if(...): ... endif;` o `foreach(...): ... endforeach;`) intercaladas con etiquetas `<div>`, `<ul>` o atributos de formulario.
Esta práctica se utiliza para automatizar la generación de elementos repetitivos (como filas de una tabla), personalizar atributos (como la clase `active` en un enlace) o condicionar la visibilidad de bloques enteros de interfaz, permitiendo que el programador mantenga la legibilidad del diseño visual mientras controla la lógica de presentación.

4. Salida de datos

- **echo** : Estructura del lenguaje (language construct) utilizada para enviar una o más cadenas de texto al flujo de salida del servidor (generalmente el cuerpo de una respuesta HTTP), siendo la instrucción de salida más utilizada en PHP debido a su alta eficiencia y velocidad, ya que no tiene valor de retorno (no devuelve nada al ser ejecutada).

Se utiliza para imprimir variables, etiquetas HTML o texto plano directamente en el documento, además de permitir pasar múltiples argumentos separados por comas

(por ejemplo echo \$a, \$b, \$c;), lo que evita la necesidad de concatenar cadenas previamente y optimiza ligeramente el uso de memoria en scripts de alto tráfico.

- **print** : Estructura del lenguaje diseñada para mostrar datos en la salida que siempre devuelve el valor entero 1 tras su ejecución exitosa, debido a este valor de retorno, print puede ser utilizado dentro de expresiones complejas o contextos lógicos donde se requiera evaluar si la operación de salida se realizó (aunque esto es poco frecuente en la práctica moderna), a diferencia de echo, print solo acepta un único argumento de cadena.

Se utiliza principalmente en escenarios de depuración básica o cuando la lógica del código exige que la instrucción se comporte como una función que retorna un valor, siendo marginalmente más lenta que echo debido a esta sobrecarga de retorno.

5. Terminación de ejecución

- **die()** : Detiene inmediatamente la ejecución del script PHP en el punto exacto donde es invocada, al activarse el motor de PHP cesa el procesamiento de cualquier código posterior, cierra las conexiones abiertas y envía una respuesta al cliente, esta instrucción acepta un parámetro opcional, que es una cadena de texto que se imprimirá antes de finalizar (útil para mensajes de error) o un número entero que actuará como código de estado de salida.

Se utiliza principalmente en el manejo de errores críticos, como fallos en la conexión a bases de datos o falta de permisos en archivos, para evitar que el script continúe operando con datos incompletos o estados inconsistentes.

- **exit()** : Interrumpe de forma inmediata la ejecución de un script, en la que su versatilidad radica en que permite definir el estado de salida mediante un parámetro opcional: si se proporciona una cadena de texto, esta se imprime justo antes de la detención, si se emplea un número entero (entre 0 y 254), este funciona como un código de retorno para el sistema operativo, donde el 0 simboliza una ejecución exitosa y otros valores indican errores específicos.

En la práctica profesional, su uso es fundamental para garantizar el control del flujo, especialmente tras realizar redirecciones con header("Location: ..."), asegurando que no se procese código residual, su nombre aporta una semántica clara de finalización lógica y controlada, lo que favorece la legibilidad y el mantenimiento del código en entornos de desarrollo.

6. Variables

- **Declaración con \$** : Las variables se declaran anteponiendo el símbolo de dólar, seguido de un nombre válido (que debe empezar por una letra o guion bajo) crea el espacio en el contenedor de variables del script, siendo fundamental para instanciar cualquier dato en la memoria volátil durante la ejecución de la petición.
- **Asignación** : Proceso de vincular un valor específico a un identificador de variable utilizando el operador de asignación simple (=), en este paso PHP evalúa la expresión situada a la derecha del operador y almacena el resultado en el espacio de memoria reservado para la variable a la izquierda, siendo una operación de copia por valor por defecto (excepto en objetos), lo que significa que el dato se duplica en la nueva variable.

Se utiliza para inicializar datos tras su declaración, permitiendo que el programa guarde resultados de funciones, entradas de usuario o valores constantes para su manipulación posterior en la lógica del negocio.

- **Reasignación** : Capacidad de modificar el contenido de una variable existente sustituyendo su valor actual por uno nuevo mediante una nueva operación de asignación, dado que PHP gestiona la memoria de forma dinámica, al reasignar una variable, el motor libera internamente el valor anterior (si no hay más referencias a él) y vincula el nuevo dato al mismo identificador.
Se utiliza en estructuras iterativas (como contadores en bucles), en la actualización de estados de objetos o en el procesamiento secuencial de datos donde una variable "evoluciona" a medida que el script avanza, permitiendo un uso eficiente de los nombres de variables sin necesidad de crear nuevos identificadores para cada paso.
- **Tipado dinámico** : Tipo de dato (string, integer, array, etc.) vinculado al valor almacenado y no al nombre de la variable, permitiendo que una variable declarada inicialmente con un número entero pueda ser reasignada con una cadena de texto o un booleano en cualquier momento sin causar errores de compilación o ejecución ya que el motor de PHP realizará una gestión automática del tipo de dato en el "zval" (la estructura interna de C que representa las variables en PHP), ajustando el contenedor según la naturaleza del nuevo valor.
Se utiliza para dotar al desarrollador de una gran flexibilidad y velocidad de prototipado, eliminando la necesidad de realizar conversiones de tipo manuales (casting) constantes.

7. Tipos de datos

- **Tipos escalares** : Unidades de información más simples y básicas que PHP puede manejar, representando un valor único en una variable (a diferencia de los tipos compuestos como arrays u objetos).
Se denominan "escalares" porque no pueden ser descompuestos en partes más pequeñas con significado independiente dentro del sistema de tipos del lenguaje, el motor de PHP optimiza el almacenamiento de estos tipos en estructuras internas llamadas zvals, permitiendo un acceso rápido y un consumo de memoria mínimo y se utilizan como los bloques de construcción primordiales para cualquier lógica de programación, desde cálculos matemáticos hasta la manipulación de texto y evaluaciones lógicas de control.

|
| — **string** : Almacena una serie de caracteres donde cada unidad se corresponde exactamente con un byte (8 bits), las cadenas pueden ser delimitadas por comillas simples (' '), que tratan el contenido como un literal puro, o comillas dobles (" "), que activan la interpolación de variables y el procesamiento de secuencias de escape (como \n o \t).
| Se utiliza para la representación de cualquier información textual, desde fragmentos de código HTML hasta la construcción de consultas SQL y manipulación de datos JSON, siendo el tipo de dato con mayor presencia en el desarrollo de interfaces web dinámicas.
|

- | — **int** : Representa números pertenecientes al conjunto de los números enteros, es decir, valores numéricos sin componentes decimales o fraccionarios, el tamaño de un entero es dependiente de la arquitectura del servidor, en sistemas de 64 bits (el estándar actual), el rango abarca desde hasta y se permite la definición de enteros en base decimal, hexadecimal (prefijo 0x), octal (prefijo 0o o 0) y binaria (prefijo 0b).
Se utiliza fundamentalmente para identificadores únicos (IDs) en bases de datos, índices de arrays, contadores en estructuras de control y cualquier operación aritmética que requiera precisión absoluta sin los errores de redondeo inherentes a los números reales.
- | — **float** : Representa valores numéricos que incluyen una parte decimal o fraccionaria, esto gracias a que PHP implementa estos números bajo el estándar IEEE 754 de precisión doble, lo que permite representar magnitudes extremadamente grandes o pequeñas, incluyendo la notación científica (por ejemplo 1.2e3).
Es crítico comprender que, debido a la naturaleza binaria de la aritmética de punto flotante, ciertos valores decimales no pueden representarse con precisión exacta (como 0.1 o 0.7), lo que puede generar discrepancias en comparaciones de igualdad directa y se utiliza en cálculos científicos, coordenadas geográficas, medidas físicas y aplicaciones financieras, aunque en estas últimas se recomienda el uso de la extensión BCMath para evitar errores de precisión.
- | — **bool** : Representa el valor de verdad más simple de la lógica computacional, pudiendo adoptar únicamente los estados true (verdadero) o false (falso), a pesar de su sencillez, es el motor que rige el flujo de ejecución del programa mediante estructuras condicionales y bucles, PHP aplica una conversión automática a booleano en contextos lógicos (conocida como falsy values), donde el número 0, el 0.0, una cadena vacía "", un array vacío [] y el valor null se interpretan como false, mientras que cualquier otro valor se considera true. Se utiliza para definir banderas de estado (flags), controlar el acceso a bloques de código y capturar el resultado de operadores de comparación y lógicos.

- **Tipos especiales**

- | — **null** : Tipo de dato único que solo puede poseer un valor: la constante insensible a mayúsculas null, representando la ausencia total de valor o una variable que no tiene contenido definido, siendo que una variable se considera NULL en tres escenarios específicos: si se le ha asignado la constante null explícitamente, si no se le ha asignado ningún valor aún tras su declaración, o si ha sido eliminada mediante la función unset().
Se utiliza fundamentalmente para inicializar variables que recibirán datos opcionales, para resetear estados de objetos y como valor de retorno en funciones que no encuentran un resultado válido (como una búsqueda fallida en una base de datos), permitiendo diferenciar entre un valor "vacío" (como 0 o "") y un valor "inexistente".

— **resource** : Variable especial que contiene un puntero o referencia a un recurso externo que no es gestionado directamente por el motor de PHP, estos recursos son creados y utilizados por extensiones específicas para manejar conexiones externas, como manejadores de archivos (fopen), conexiones a bases de datos (mysqli_connect) o lienzo de procesamiento de imágenes (imagecreatetruecolor) y dado que son punteros a estructuras externas en memoria C, no pueden ser convertidos a otros tipos ni serializados. Se utilizan para interactuar con el sistema operativo y servicios de terceros, donde PHP actúa como un puente de comunicación, liberando automáticamente el recurso mediante el recolector de basura (Garbage Collector) cuando la variable ya no tiene referencias activas.

— **callable** : "Pseudo-tipo" o type hint que indica que una variable puede ser invocada como una función, esto incluye nombres de funciones almacenados en strings ('miFuncion'), arreglos que referencian métodos de clase ([\$objeto, 'metodo']), funciones anónimas (clausuras/closures) o objetos que implementan el método mágico __invoke() y a diferencia de otros tipos, no se puede "forzar" un valor a ser callable mediante casting, sino que debe cumplir con la estructura que el motor de PHP requiere para ejecutarlo. Se utiliza extensivamente en la programación funcional y patrones de diseño (como Callback o Strategy), permitiendo pasar lógica de ejecución completa como argumento a otras funciones (por ejemplo, en array_map o usort) para aumentar la modularidad y flexibilidad del código.

- **Tipos compuestos**

— **array** : Estructura de datos que relaciona valores con claves, que a diferencia de otros lenguajes, los arrays de PHP son extremadamente versátiles, ya que pueden funcionar simultáneamente como listas indexadas (con claves numéricas), tablas hash o diccionarios (con claves de tipo string), pilas (stacks) o colas (queues). Se utiliza para gestionar colecciones de datos relacionados, como los resultados de una consulta a base de datos, listas de configuración o estructuras de respuesta JSON, siendo la herramienta principal para la manipulación masiva de información dentro de un script.

— **object** : Representa una entidad que encapsula tanto estado (propiedades/variables) como comportamiento (métodos/funciones), que a diferencia de los tipos escalares que se pasan por valor, los objetos se manejan mediante referencias: cuando se asigna un objeto a una nueva variable, ambas apuntan a la misma instancia en memoria. Permiten implementar los pilares de la Programación Orientada a Objetos (POO), como el encapsulamiento, la herencia y el polimorfismo y se utilizan para modelar entidades del mundo real o abstracciones lógicas del sistema (como un Usuario, un Pedido o un Controlador), permitiendo una organización del código mucho más modular, reutilizable y escalable en aplicaciones de gran envergadura.

8. Conversión de tipos

- **Conversión automática** : Conocida técnicamente en PHP como Type Juggling, es el proceso mediante el cual el motor de PHP cambia el tipo de dato de una variable de forma transparente y sin intervención del programador, este fenómeno ocurre típicamente cuando se utiliza una variable en un contexto que requiere un tipo diferente al que posee originalmente, por ejemplo al usar un string que contiene un número en una operación matemática, PHP lo convertirá internamente a int o float para completar el cálculo.
Se utiliza para simplificar el flujo de desarrollo al permitir que el lenguaje "adivine" la intención del programador, aunque en aplicaciones complejas se debe vigilar para evitar resultados inesperados en comparaciones lógicas.
- **Casting explícito** : Acción manual donde el desarrollador fuerza una variable a convertirse en un tipo específico utilizando un operador de conversión, esto se logra anteponiendo el nombre del tipo deseado entre paréntesis antes de la variable (por ejemplo (int)\$valor, (string)\$id o (bool)\$estado), a diferencia de la conversión automática, el casting es una instrucción deliberada que garantiza que el dato tendrá la naturaleza requerida antes de ser procesado por una función o estructura de control.
Se utiliza para asegurar la integridad de los datos, limpiar entradas de usuario (como convertir un parámetro de URL en un entero seguro) y cumplir con contratos de tipos en firmas de funciones cuando no se usa el modo estricto.
- **Coerción de tipos** : Ocurre estrictamente durante operaciones de evaluación, como comparaciones entre tipos dispares o el paso de argumentos a funciones, aquí PHP "fuerza" (coacciona) a uno de los operandos a transformarse para que la operación sea semánticamente posible, por ejemplo en una comparación de igualdad no estricta (==), si comparas el entero 0 con la cadena "0", PHP coacciona ambos a un tipo común para que la comparación resulte en true.
Se utiliza como un mecanismo de flexibilidad del lenguaje, pero representa uno de los puntos más críticos de seguridad y lógica, razón por la cual se introdujo el modo strict_types para limitar este comportamiento en aplicaciones que requieren alta precisión.

9. Constantes

- **define()** : Declara constantes en tiempo de ejecución y al ser una función, permite que el nombre de la constante sea el resultado de una expresión o una variable, y puede invocarse dentro de estructuras de control como un if, las constantes definidas con este método tienen un alcance global automático, lo que significa que son accesibles desde cualquier parte del script (incluyendo dentro de funciones o métodos) sin necesidad de usar la palabra clave global.
Se utiliza principalmente para configuraciones dinámicas de la aplicación, como definir rutas de directorios basadas en el entorno del servidor o cargar credenciales de bases de datos desde archivos de configuración externos.
- **const** : Define constantes en tiempo de compilación y que a diferencia de define(), su uso es obligatorio cuando se declaran constantes dentro de una clase (constantes de clase), y no puede utilizarse dentro de bloques condicionales o

funciones, ya que el motor Zend debe conocer su valor antes de ejecutar el script, es marginalmente más rápida que `define()` porque no implica la sobrecarga de una llamada a función. Se utiliza para definir valores estáticos y permanentes que no dependen de la lógica del programa, como versiones de la aplicación, códigos de error estándar o factores de conversión matemáticos, promoviendo un código más limpio y legible bajo los estándares modernos de programación orientada a objetos.

10. Constantes mágicas

- **__FILE__** : Devuelve la ruta absoluta completa, incluyendo el nombre del archivo, del script PHP que se está ejecutando actualmente y a diferencia de las constantes normales, su valor no es estático en toda la aplicación, sino que se resuelve en tiempo de ejecución dependiendo del archivo donde se invoque: si se usa dentro de un archivo incluido (vía `include` o `require`), devolverá la ruta de ese archivo específico y no la del script principal.
Se utiliza principalmente para realizar operaciones de diagnóstico, registrar errores detallados en archivos de log o como base para construir rutas dinámicas a otros recursos del servidor, garantizando que la aplicación sea portable independientemente de la estructura de directorios del host.
- **__DIR__** : Devuelve la ruta absoluta del directorio donde se encuentra el archivo actual, siendo el equivalente a ejecutar `dirname(__FILE__)`, esta constante es una de las herramientas más potentes para la gestión de dependencias y la inclusión de archivos, ya que permite definir rutas relativas al archivo actual que se transforman en rutas absolutas para el sistema operativo, evitando los problemas comunes de configuración del `include_path`.
Se utiliza de forma extensiva en cargadores automáticos (autoloader), para incluir archivos de configuración, plantillas o clases (`require __DIR__ . '/config.php'`), asegurando que el script siempre localice sus recursos sin importar desde qué carpeta se haya iniciado la ejecución.
- **__LINE__** : Devuelve el número de línea actual dentro del archivo de código fuente donde se encuentra escrita, cambiando su valor dinámicamente según la posición física de la instrucción en el editor de texto, siendo una herramienta indispensable para la depuración (debugging) y el manejo de excepciones, ya que permite al desarrollador identificar con precisión quirúrgica el punto exacto donde ocurrió un evento o un fallo.
Se utiliza frecuentemente en la creación de sistemas de log personalizados y en el lanzamiento de excepciones (`throw new Exception("Error en línea " . __LINE__)`), facilitando el mantenimiento y la corrección de errores en aplicaciones con bases de código extensas.

11. Operadores

• Aritméticos

|
| — **+** : Realiza la suma matemática entre dos operandos, si ambos operandos son
| números enteros (int), el resultado será un entero y si al menos uno es de
| punto flotante (float), el resultado será un flotante, debido a la naturaleza de
| PHP si se intenta sumar tipos no numéricos (como un string que contiene un

número), el motor realizará una coerción de tipos automática antes de la operación.

Se utiliza para cálculos incrementales, acumulación de totales en estructuras de datos y aritmética básica de negocio, siendo el operador fundamental para la gestión de contadores y sumatorias.

— **-** : Resta el operando de la derecha del operando de la izquierda, si ambos operandos son números enteros (int), el resultado será un entero y si al menos uno es de punto flotante (float), el resultado será un flotante, este operador también tiene una variante unaria (ej. -\$a) que se utiliza para invertir el signo de un número o convertir un valor positivo en negativo.

Se emplea comúnmente para calcular diferencias entre valores, aplicar descuentos en aplicaciones de comercio electrónico y determinar distancias o deltas entre índices numéricos.

— ***** : Calcula el producto de dos números, en la que PHP maneja automáticamente el desbordamiento de enteros (integer overflow): si el resultado de una multiplicación de enteros excede el límite de bits del sistema (32 o 64 bits), el motor convertirá el resultado automáticamente a tipo float para preservar la magnitud del valor.

Se utiliza para el escalado de datos, cálculos de impuestos o tasas, y en algoritmos que requieren transformaciones proporcionales de variables numéricas.

— **/** : Realiza el cociente entre el dividendo (izquierda) y el divisor (derecha), en la que a diferencia de otros lenguajes donde la división de dos enteros puede devolver un entero truncado, en PHP la división siempre devuelve un float a menos que ambos operandos sean enteros y el resultado sea una división exacta sin resto.

Es crítico validar que el divisor no sea cero, ya que esto lanzará una excepción `DivisionByZeroError` en versiones modernas y se utiliza para promedios, distribución de elementos en paginación y cualquier reparto proporcional de recursos.

— **%** : Devuelve el resto o residuo de una división entera, en la que antes de realizar la operación, PHP convierte ambos operandos a tipo int (truncando cualquier parte decimal si existen flotantes), además el signo del resultado siempre es el mismo que el del dividendo.

Se utiliza frecuentemente en lógica de programación para determinar la paridad de un número (par/impar), implementar ciclos rotativos (como colores alternos en filas de una tabla) o verificar la divisibilidad exacta entre dos magnitudes.

— ****** : Introducido en PHP 5.6, eleva el operando de la izquierda a la potencia del operando de la derecha (base y exponente), siendo asociativo hacia la derecha (por ejemplo $2 ** 3 ** 2$ se evalúa como $2 ** (3 ** 2)$), proporcionando una sintaxis mucho más limpia y eficiente que la función tradicional `pow()`.

Se utiliza en cálculos financieros de interés compuesto, algoritmos

criptográficos, progresiones geométricas y cualquier fórmula científica que requiera potencias de valores.

- **Comparación**

— **==** : Compara dos valores tras realizar una coerción de tipos si es necesario, esto significa que si los operandos son de tipos diferentes (por ejemplo, un int y un string), PHP intentará convertirlos a un tipo común antes de realizar la comparación lógica, por ejemplo, la expresión `0 == "0"` devolverá `true`. Se utiliza en contextos donde la procedencia de los datos puede ser variada (como entradas de formularios que siempre llegan como strings) y solo interesa validar el valor semántico, aunque su uso requiere precaución para evitar falsos positivos en comparaciones críticas de seguridad.

— **!=** : Devuelve `true` si los valores de los operandos no son iguales después de aplicar la conversión automática de tipos, ignorando las diferencias de tipo de dato si el valor representado es el mismo (por ejemplo, `5 != "5"` devolverá `false`), siendo el opuesto lógico de `==`. Se utiliza para validar que una variable ha cambiado su estado inicial, para filtrar elementos en colecciones de datos o para asegurar que una entrada de usuario no coincide con un valor prohibido o por defecto.

— **<** : Compara el operando de la izquierda con el de la derecha, devolviendo `true` solo si el primero es estrictamente inferior al segundo, si se comparan strings, PHP utiliza el orden lexicográfico basado en los valores de los bytes de los caracteres y si se comparan tipos mixtos (como un número y un string numérico), se realiza una conversión a número. Se utiliza habitualmente en estructuras de control de bucles (como `for`), para validar rangos de edad, límites de stock o para ordenar elementos de forma ascendente en algoritmos de clasificación.

— **>** : Compara el operando de la derecha con el de la izquierda, devolviendo `true` solo si el primero es estrictamente inferior al segundo, si se comparan strings, PHP utiliza el orden lexicográfico basado en los valores de los bytes de los caracteres y si se comparan tipos mixtos (como un número y un string numérico), se realiza una conversión a número. Se utiliza habitualmente en estructuras de control de bucles (como `for`), para validar rangos de edad, límites de stock o para ordenar elementos de forma ascendente en algoritmos de clasificación.

— **=** : Operador de asignación cuya función no es evaluar una igualdad, sino transferir el valor de la expresión de la derecha a la variable situada a la izquierda, en el contexto de una estructura condicional (como un `if`), usar `=` por error en lugar de `==` resultará en una asignación exitosa que casi siempre se evaluará como `true`, provocando fallos lógicos graves. Se utiliza exclusivamente para inicializar variables, actualizar estados de objetos o capturar el retorno de funciones en una dirección de memoria específica.

— **>=** : Combina la lógica de comparación de magnitud con la de igualdad, devolviendo true si el operando de la derecha es menor que el de la izquierda o si ambos son iguales tras la conversión de tipos, siendo un operador inclusivo y se utiliza de manera extensiva en la validación de límites máximos permitidos (como el tamaño de un archivo o la longitud de un texto) y en iteraciones donde el último índice de una lista también debe ser procesado por la lógica del bucle.

— **<=** : Combina la lógica de comparación de magnitud con la de igualdad, devolviendo true si el operando de la izquierda es menor que el de la derecha o si ambos son iguales tras la conversión de tipos, siendo un operador inclusivo y se utiliza de manera extensiva en la validación de límites máximos permitidos (como el tamaño de un archivo o la longitud de un texto) y en iteraciones donde el último índice de una lista también debe ser procesado por la lógica del bucle.

- **Comparación estricta**

— **===** : Operador de identidad estricta (o "triple igual") que compara dos operandos sin realizar ninguna coerción de tipos previa, para que esta expresión devuelva true, tanto el valor como el tipo de dato de ambos operandos deben ser exactamente iguales (por ejemplo, `5 === 5` es verdadero, pero `5 === "5"` es falso), siendo el estándar de oro en el desarrollo profesional de PHP, ya que elimina las ambigüedades del tipado débil y evita resultados inesperados en validaciones críticas.

Se utiliza obligatoriamente para verificar valores de retorno de funciones que pueden devolver tipos mixtos, como `strpos()`, que devuelve 0 (posición) o false (no encontrado), permitiendo distinguir entre un índice válido y un fallo lógico.

— **!==** : Devuelve true si los operandos no son iguales en valor o si no son del mismo tipo de dato, evitando la conversión automática de tipos de PHP, y proporcionando una comparación determinista y segura.
Se utiliza para asegurar que una variable es distinta de un estado específico de forma absoluta, siendo fundamental en estructuras condicionales donde se requiere confirmar que una operación no solo falló, sino que falló con un tipo de dato específico (por ejemplo, `if ($resultado !== null)`), garantizando que el flujo del programa no se desvíe por una interpretación errónea de valores equivalentes como 0 o cadenas vacías.

- **Lógicos**

— **and** : Realiza una conjunción lógica entre dos expresiones, devolviendo true únicamente si ambos operandos se evalúan como verdaderos, aunque funcionalmente es similar a `&&`, su principal característica es que posee una precedencia muy baja, incluso menor que el operador de asignación (`=`), debido a esto, una instrucción como `$resultado = true and false`, asignará true a la variable y luego evaluará el `and`.

- | Se utiliza principalmente en flujos de control de estilo "decapitado" o en combinación con otros operadores de baja prioridad, aunque en la programación moderna ha sido desplazado por `&&` para evitar confusiones de precedencia.
- |
- | `or` : Devuelve true si al menos uno de los dos operandos es verdadero, teniendo una precedencia extremadamente baja, esta característica lo hace útil para patrones de ejecución condicional antiguos conocidos como "short-circuit execution" (por ejemplo `abrir_archivo()` or `die();`), donde la segunda parte solo se ejecuta si la primera falla (devuelve false).
En la lógica de negocio estándar de un `if`, se prefiere el uso de `||` para asegurar que la evaluación de los términos ocurra antes que otras operaciones de mayor jerarquía.
- |
- | `!` : Operador unario que invierte el valor lógico de su único operando, transformando true en false y viceversa, y si el operando no es un booleano, PHP realizará una conversión automática a booleano antes de aplicar la inversión (por ejemplo, `!0` devuelve true porque 0 se evalúa como false).
Se utiliza para verificar la ausencia de condiciones, validar que una función ha fallado, comprobar si una variable está vacía o simplemente para alternar estados binarios dentro de la lógica del programa.
- |
- | `&&` : Realiza una conjunción lógica con una precedencia alta, lo que garantiza que se evalúe antes que la mayoría de los operadores de asignación o de comparación simple, y utiliza cortocircuito, es decir si el primer operando es false, el segundo ni siquiera se evalúa, optimizando así el rendimiento.
Es el estándar de facto en el desarrollo de aplicaciones para combinar múltiples condiciones dentro de estructuras `if` o `while`, asegurando que la expresión completa sea evaluada de forma predecible y segura.
- |
- | `||` : Realiza una disyunción lógica con una precedencia superior a su variante `or`, devolviendo true si cualquiera de los operandos es verdadero y también emplea cortocircuito, es decir si el primer operando es true, el motor de PHP no procesa el segundo, ya que el resultado final de la expresión está garantizado.
Se utiliza para definir múltiples escenarios válidos en una sola validación (por ejemplo, `if ($rol == 'admin' || $rol == 'editor')`), permitiendo que el flujo de ejecución continúe si se cumple al menos uno de los requisitos establecidos.

● Asignación

- | `+=` : Combina la suma de un valor del operando derecho al valor actual de la variable izquierda y, acto seguido, asigna el resultado de esa suma a la misma variable, siendo una forma abreviada y más eficiente de escribir `$a = $a + $b`, si la variable original no estaba definida, PHP emitirá un aviso (Warning) e intentará tratarla como 0 antes de sumar.
Se utiliza extensivamente en la creación de acumuladores de totales (como el carrito de compras), en el incremento de contadores dentro de bucles y en la

construcción progresiva de valores numéricos.

— **-=** : Resta el valor del operando derecho al valor contenido en la variable de la izquierda, guardando el nuevo resultado en esta última, siendo una simplificación sintáctica de $\$a = \$a - \$b$, respetando las reglas de coerción de tipos, por lo que si se aplica sobre un string numérico, PHP lo convertirá a un número antes de restar.

Se utiliza habitualmente para gestionar decrementos de inventario, reducir saldos en cuentas de usuario o ajustar límites de intentos en sistemas de seguridad y validación.

— ***=** : Multiplica el valor actual de la variable por el operando de la derecha y actualiza la variable con el producto obtenido y si el resultado de la operación excede la capacidad de un entero (integer overflow), PHP convertirá automáticamente el tipo de la variable a float para evitar la pérdida de datos. Se utiliza en algoritmos de escalado, cálculos de intereses compuestos, aplicaciones de impuestos o tasas porcentuales y en cualquier proceso donde un valor deba ser transformado mediante un factor de escala constante.

— **/=** : Divide el valor almacenado en la variable por el operando derecho, asignando el cociente resultante a la propia variable, es importante notar que en PHP, esta operación casi siempre resultará en un cambio del tipo de dato de la variable a float, a menos que la división sea exacta y ambos operandos sean enteros, además es imperativo asegurar que el operando derecho sea distinto de cero para evitar un error fatal de ejecución. Se utiliza para normalizar valores, calcular promedios ponderados y en la distribución equitativa de recursos dentro de una lógica de iteración.

- **Operadores modernos**

— **null coalescing ??** : Devuelve su primer operando si este existe y no es null, y en caso contrario devuelve el segundo operando, todo esto sin emitir un aviso (Notice) si la variable de la izquierda no ha sido declarada, lo que lo convierte en la herramienta perfecta para gestionar datos externos.

Se utiliza principalmente para asignar valores por defecto a variables que provienen de arreglos superglobales (como `$_GET` o `$_POST`) o de bases de datos, permitiendo escribir instrucciones como `$usuario = $_GET['id'] ?? 'invitado'`; de forma segura y concisa.

— **null coalescing assignment ??=** : Introducido en PHP 7.4, combina la lógica de la fusión de nulo con la asignación, verificando si la variable situada a la izquierda es null (o no existe) y si se cumple esa condición le asigna el valor de la expresión de la derecha, pero si la variable ya tiene un valor distinto de null, la instrucción no realiza ninguna acción.

Se utiliza para la inicialización diferida (lazy initialization) de propiedades de objetos o configuraciones, evitando la redundancia de código como `$a = $a ?? $b`; y manteniendo la integridad de los datos preexistentes con una sintaxis mínima.

- **spaceship <=>** : Devuelve un entero (int) basándose en la relación entre dos operandos, devolviendo 0 si ambos son iguales, 1 si el de la izquierda es mayor, y -1 si el de la derecha es mayor, realizando una comparación completa (menor, igual y mayor) en una sola instrucción y siguiendo las reglas de comparación estándar de PHP para diferentes tipos de datos.
Se utiliza casi exclusivamente como función de retorno en algoritmos de ordenación personalizada (como usort()), simplificando drásticamente la lógica necesaria para clasificar colecciones de objetos o arreglos complejos.
- **ternario ? :** : Forma compacta de escribir una estructura “if-else” que devuelve un valor basado en una condición booleana, permitiendo que una variante abreviada conocida como “Elvis operator” (?:), devuelva el primer operando si este se evalúa como true (en contexto booleano) y el segundo si es false.
Se utiliza para asignaciones condicionales simples en una sola línea, como definir clases CSS dinámicas o mensajes de estado, mejorando la legibilidad del código siempre que la lógica evaluada no sea excesivamente compleja.

- **Otros operadores**

- **Concatenación** : Representado por un punto (.), es el mecanismo utilizado para combinar dos o más cadenas de caracteres en una sola secuencia y a diferencia de otros lenguajes que utilizan el signo + (lo que puede generar ambigüedad con operaciones aritméticas), PHP reserva el punto exclusivamente para la unión de strings, si uno de los operandos no es una cadena (como un int o un float), PHP realiza una coerción de tipos automática a string antes de unirlos.
Se utiliza para construir mensajes dinámicos, generar bloques de código HTML, estructurar consultas SQL complejas y ensamblar rutas de archivos combinando directorios y nombres de recursos.
- **Operador de Asignación sobre Concatenación (.=)** : Añade el valor de la expresión de la derecha al final del contenido actual de la variable situada a la izquierda, siendo la forma abreviada de escribir \$a = \$a . \$b.
Este operador es extremadamente útil para la construcción progresiva de datos, permitiendo acumular fragmentos de texto, filas de una tabla o partes de un cuerpo de correo electrónico a lo largo de un ciclo o estructura lógica y se utiliza para optimizar la legibilidad del código cuando se trabaja con cadenas extensas, evitando la redundancia de nombrar la variable destino en cada paso de la construcción.
- **instanceof** : Determina si una variable es una instancia de una clase específica, si hereda de una clase determinada o si implementa una interfaz concreta, devolviendo un valor booleano (true o false) tras realizar la comprobación en tiempo de ejecución y a diferencia de las funciones de tipo escalar, instanceof es fundamental en la programación orientada a objetos para validar la integridad de los datos antes de invocar métodos específicos de usuario o determinar la respuesta adecuada según el código de estado de una

12. Estructuras de control

- **Condicionales**

- **if** : Ejecuta un bloque de código únicamente si una expresión evaluada se resuelve como true, si la condición es falsa el motor de PHP ignora por completo el bloque contenido entre las llaves {} y continúa con la ejecución de la instrucción inmediata posterior. Se utiliza para validar requisitos mínimos de ejecución, como verificar si una variable ha sido enviada o si un usuario tiene los permisos necesarios para acceder a un recurso específico.

- **elseif** : Encadena múltiples condiciones de forma secuencial tras un if inicial, en el que el motor de PHP evalúa estas condiciones en orden de aparición y ejecuta exclusivamente el primer bloque cuya expresión sea verdadera, una vez que se cumple una condición, todas las subsiguientes (incluyendo el else final) son omitidas, optimizando así el rendimiento del script.

- Se utiliza para manejar casos de error por defecto, mostrar mensajes de denegación de acceso o establecer valores de inicialización cuando no se cumplen los criterios de búsqueda principales, aunque su uso opcional se considera una buena práctica para garantizar que el programa mantenga un estado predecible y responda de manera controlada ante datos inesperados.

- **else** : Define un bloque de código por defecto que se ejecutará únicamente si todas las condiciones previas (if y todos los elseif) se han evaluado. Se utiliza habitualmente en el manejo de excepciones (catch (Exception \$e)), en patrones de diseño para asegurar que un objeto cumple con un contrato esperado y para prevenir errores fatales al intentar operar con objetos de tipos incompatibles.

- **switch** : Ejecuta diferentes bloques de código basándose en el valor de una única expresión, que a diferencia de una cadena de if/elseif, el motor de PHP evalúa la expresión una sola vez y busca la coincidencia dentro de las cláusulas case, es fundamental comprender que switch utiliza comparación débil (==), lo que puede generar coincidencias inesperadas debido a la coerción de tipos, cada bloque case requiere la instrucción break; para detener la ejecución; de lo contrario, el flujo continuará hacia el siguiente caso (comportamiento conocido como fall-through). Se utiliza para organizar lógicas con múltiples opciones discretas, como procesar comandos de una interfaz o determinar acciones según el estado de un pedido.

- **match (PHP 8)** : Evolución moderna y más potente de switch, que a diferencia de su predecesor, match devuelve un valor directamente, utilizando una comparación estricta (===) y sin requerir la instrucción break, ya que solo ejecuta el brazo que coincide exactamente, además, match es exhaustivo: si el valor evaluado no coincide con ninguna opción y no se define un brazo default, el motor de PHP lanzará un error fatal (UnhandledMatchError).

Se utiliza para realizar asignaciones condicionales directas y seguras, reduciendo drásticamente la verbosidad del código y eliminando errores comunes de lógica derivados del tipado débil o el olvido de interrupciones de flujo.

13. Bucles

- **while** : Ejecuta un bloque de código repetidamente mientras una condición específica se evalúe como true, el motor de PHP verifica la condición al inicio de cada ciclo y si la expresión es falsa desde el primer intento, el bloque de código interno nunca llega a ejecutarse, dado que no posee un contador intrínseco, es responsabilidad del desarrollador modificar las variables involucradas en la condición dentro del cuerpo del bucle para evitar ciclos infinitos que agoten la memoria del servidor.

Se utiliza principalmente en escenarios donde no se conoce de antemano el número exacto de iteraciones necesarias, como la lectura de registros de un puntero de base de datos o el procesamiento de datos hasta alcanzar el final de un archivo (EOF).

- **do while** : Variante de while cuya característica distintiva es que la condición se evalúa al final de cada iteración, en lugar de al principio, esto garantiza que el bloque de código interno se ejecute al menos una vez, independientemente de si la condición es verdadera o falsa inicialmente y al igual que el while, requiere una lógica interna que eventualmente invalide la condición de permanencia para permitir la salida del bucle.

Se utiliza en situaciones donde la primera ejecución es necesaria para obtener los datos que luego serán evaluados, como en la generación de interfaces interactivas, validaciones de entrada de usuario que deben ocurrir antes de la comprobación, o algoritmos que requieren un paso inicial de configuración.

- **for** : Bucle ideal para casos donde el número de repeticiones es conocido de antemano, cuya sintaxis se divide en tres expresiones separadas por puntos y coma: la inicialización (ejecutada una vez al principio), la condición (evaluada antes de cada iteración) y el incremento/paso (ejecutado al final de cada ciclo), esta organización centraliza el control del bucle en una sola línea, reduciendo el riesgo de errores lógicos y bucles infinitos.

Se utiliza de forma extensiva para recorrer arreglos indexados numéricamente, ejecutar cálculos matemáticos en series progresivas y realizar tareas de procesamiento por lotes con un límite definido de elementos.

- **foreach** : Bucle diseñada exclusivamente para iterar sobre arrays y objetos que implementan la interfaz Traversable, que a diferencia de los bucles anteriores, foreach no depende de un contador manual, sino que el motor de PHP gestiona internamente el puntero del arreglo, extrayendo automáticamente la clave y el valor de cada elemento en cada paso del ciclo, caracterizándose por ser más segura y legible, ya que evita errores de desbordamiento de índice (out-of-bounds) y trabaja sobre una copia de los datos (o referencias si se especifica).

Se utiliza como el estándar de la industria para renderizar listas dinámicas en HTML, procesar colecciones de objetos provenientes de un ORM y transformar estructuras de datos complejas de manera eficiente.

14. Arrays

• Tipos de arrays

- | — **Indexados** : Array en la que cada elemento se le asigna automáticamente un índice numérico entero, por defecto el motor de PHP comienza la indexación en el número 0 e incrementa este valor secuencialmente para cada nuevo elemento añadido, internamente PHP mantiene estos índices en una tabla hash ordenada, lo que permite un acceso extremadamente rápido a los datos mediante su posición.
| Se utilizan principalmente para almacenar listas de elementos homogéneos donde el orden de aparición es relevante, como una lista de nombres de archivos, una serie de resultados de una encuesta o los registros de una columna específica de una base de datos.
- | — **Asociativos** : Estructuras de datos que utilizan claves personalizadas (generalmente de tipo string) en lugar de índices numéricos para identificar y acceder a sus valores, en este modelo el desarrollador define una relación explícita de "clave => valor", transformando el arreglo en un mapa conceptual o diccionario, en PHP permite mezclar claves numéricas y de texto en el mismo arreglo, aunque no es una práctica recomendada por legibilidad. Se utilizan para representar entidades con propiedades con nombre, como la configuración de una aplicación ('db_host' => 'localhost'), los datos de un perfil de usuario ('nombre' => 'Juan') o las cabeceras de una respuesta HTTP, facilitando la recuperación de información mediante identificadores semánticos en lugar de posiciones arbitrarias.
- | — **Multidimensionales** : Arreglo que contiene uno o más arreglos como elementos de su propia estructura, permitiendo crear matrices de dos, tres o más dimensiones (arrays de arrays) y para acceder a los datos internos, se encadenan los corchetes de los índices respectivos (por ejemplo \$matriz[0]['clave']).
| Esta estructura se utiliza para modelar datos complejos y jerárquicos, como tablas de bases de datos completas (donde el primer nivel representa las filas y el segundo las columnas), representaciones de coordenadas espaciales, estructuras de menús anidados o datos transformados desde formatos como JSON y XML para su procesamiento lógico en el servidor.

• Operaciones

- | — **Acceso a elementos** : Mecanismo mediante el cual se recupera un valor específico de un array utilizando su clave o índice dentro de corchetes [], en arrays indexados el acceso se realiza mediante un número entero (comenzando desde 0), mientras que en arrays asociativos se utiliza la cadena de texto definida como clave.
| Es una operación de tiempo constante, lo que garantiza un rendimiento óptimo independientemente del tamaño del array y se utiliza para extraer datos puntuales, realizar validaciones de existencia mediante isset() o empty(), y como paso previo a la transferencia de información hacia otras variables o

funciones.

— **Modificación** : Altera el contenido de un elemento existente o añadir uno nuevo a la estructura del array mediante el operador de asignación =, si la clave especificada ya existe, el motor de PHP sobrescribe el valor anterior con el nuevo u si la clave no existe el elemento se añade automáticamente al final del array.

En PHP permite también la sintaxis de corchetes vacíos "\$array[] = \$valor;", que añade un elemento al final asignándole el siguiente índice entero disponible y esta operación se utiliza para actualizar estados (como cambiar el estatus de un pedido), acumular datos dinámicamente durante la ejecución del script y transformar estructuras de datos crudas en información procesada para la capa de presentación.

— **Recorrido con foreach** : Técnica estándar y más eficiente para iterar sobre todos los elementos de un array sin necesidad de gestionar contadores manuales, creando una copia interna del puntero del array (o utiliza una referencia si se indica explícitamente con &) y recorriendo secuencialmente cada par clave => valor, a diferencia de un bucle for tradicional, foreach es inherentemente seguro contra errores de índice fuera de rango (out-of-bounds) y funciona de manera idéntica tanto para arrays indexados como asociativos. Se utiliza para generar listas dinámicas en HTML, aplicar transformaciones masivas a colecciones de objetos, filtrar datos bajo condiciones específicas y serializar información para respuestas de API en formato JSON.

15. Funciones

- **¿Qué es?** : Realizado mediante la palabra clave "function", seguida de un identificador único y un bloque de código delimitado por llaves {}, al definir una función, el desarrollador encapsula una lógica específica que puede ser invocada múltiples veces desde cualquier punto del script, promoviendo el principio DRY (Don't Repeat Yourself), los nombres de las funciones en PHP son insensibles a mayúsculas y minúsculas (case-insensitive), aunque la convención estándar (PSR-12) dicta el uso de camelCase.
Se utiliza para segmentar aplicaciones complejas en tareas pequeñas, legibles y fáciles de testear, permitiendo que el motor de PHP cargue la lógica en memoria una sola vez para su uso recurrente durante el ciclo de vida de la petición.
- **Parámetros** : Variables locales definidas en la firma de la función que actúan como receptores de información externa necesaria para su ejecución, permitiendo definir parámetros con valores por defecto (por ejemplo \$param = 0), lo que los convierte en opcionales durante la llamada, además desde PHP 7, se pueden aplicar declaraciones de tipo (type hinting) para restringir que los argumentos pasados sean de una naturaleza específica (como int, string o una Class).
Se utilizan para dotar a las funciones de dinamismo, permitiendo que una misma lógica procese diferentes datos de entrada, y sirven como contrato de comunicación entre el código que invoca la función y la implementación interna de la misma.

- **return** : Instrucción que finaliza la ejecución de una función y envía un valor de vuelta al punto donde fue invocada, pudiendo devolver cualquier tipo de dato, incluyendo arrays y objetos y si no se especifica un return, la función devolverá implícitamente null.

En versiones modernas de PHP, es posible definir el tipo de retorno en la firma de la función (por ejemplo `function(): int`) para garantizar que la salida cumpla con las expectativas del programa, se utiliza para transformar datos de entrada en resultados útiles, capturar estados de éxito o error tras una operación compleja y permitir el encadenamiento de funciones en expresiones lógicas más amplias.

16. Funciones esenciales

- **count()** : Contabiliza el número de elementos contenidos en un array o en cualquier objeto que implemente la interfaz Countable, si el argumento proporcionado es null, la función devolverá 0, y si se pasa un valor escalar (como un int o string), devolverá 1 (aunque en versiones modernas de PHP esto genera un aviso de tipo), posee un segundo parámetro opcional "COUNT_RECURSIVE", que permite contar todos los elementos en arrays multidimensionales de forma anidada.

Se utiliza de manera ubicua para controlar condiciones de salida en bucles for, validar si una consulta a la base de datos ha retornado resultados y verificar la integridad de colecciones de datos antes de intentar procesarlas.

- **trim()** : Elimina los espacios en blanco (u otros caracteres específicos) del principio y del final de una cadena de texto, por defecto esta función remueve espacios, tabulaciones, saltos de línea, retornos de carro y bytes nulos, permitiendo un segundo parámetro opcional donde el desarrollador puede definir una lista personalizada de caracteres a eliminar (máscara de caracteres).

Es una herramienta crítica en la etapa de saneamiento de datos (data sanitization), utilizándose sistemáticamente para limpiar entradas provenientes de formularios de usuario, normalizar datos antes de almacenarlos en una base de datos o asegurar que las comparaciones entre cadenas no fallen debido a caracteres invisibles accidentales.

17. Ámbito de variables

- **local** : Referido a las variables que son declaradas dentro del cuerpo de una función o método, estas variables son completamente invisibles y privadas para cualquier código que se encuentre fuera de dicha función y nacen en el momento en que la función es invocada y son destruidas automáticamente por el recolector de basura (Garbage Collector) una vez que la ejecución de la función finaliza y devuelve el control al programa principal.

Se utiliza para garantizar el encapsulamiento de la lógica, evitando colisiones de nombres entre diferentes partes de la aplicación y asegurando que las operaciones internas de una función no afecten accidentalmente el estado global del sistema.

- **global** : Comprende las variables definidas fuera de cualquier función o clase, situándose en el nivel superior del script, por defecto estas variables no son accesibles directamente dentro de una función debido a las reglas de aislamiento de PHP, para utilizarlas en un contexto local, se debe emplear la palabra clave global (ej. `global $variable;`) o acceder a través del array superglobal `$GLOBALS`.

Se utiliza para almacenar configuraciones que deben estar disponibles en múltiples capas del programa, como objetos de conexión a bases de datos, preferencias de idioma del usuario o constantes de entorno que rigen el comportamiento general del servidor durante la petición.

- **static** : Variable local que posee una característica especial de persistencia: a diferencia de las variables locales comunes, una variable estática no pierde su valor cuando la función termina su ejecución, el motor de PHP inicializa la variable solo la primera vez que se llama a la función y mantiene su último estado en memoria para todas las llamadas subsiguientes durante el mismo ciclo de solicitud (request). Se utiliza para implementar patrones de diseño como el Singleton, crear contadores internos que rastrean cuántas veces se ha ejecutado un proceso, o cachear resultados de cálculos costosos dentro de una función para mejorar el rendimiento sin recurrir a variables globales.

18. Inclusión de archivos

- **include** : Inserta y evalúa el contenido de un archivo externo dentro del script en ejecución, si el archivo no se encuentra, el motor emite una advertencia (*Warning*) pero continúa ejecutando el resto del programa de forma normal. Se utiliza específicamente para maquetar secciones visuales repetitivas o secundarias que no alteran la lógica vital del sistema, como barras laterales, menús de navegación inferiores o banners publicitarios.
- **include_once** : Verifica previamente si el archivo externo ya ha sido incorporado en alguna línea anterior del flujo de ejecución para evitar duplicaciones, si el archivo no existe, emite una advertencia (*Warning*) y el programa continúa. Se utiliza exclusivamente para incorporar archivos con funciones auxiliares personalizadas o librerías ligeras en sistemas complejos, asegurando que no se intente redefinir la misma función por error si el script es llamado en múltiples subarchivos conectados.
- **require** : Incorpora un archivo externo de manera obligatoria; si el sistema operativo no localiza el archivo en la ruta indicada, el motor detiene inmediatamente el procesamiento del software y genera un error fatal (*Fatal Error*). Se utiliza estrictamente para cargar componentes críticos del backend sin los cuales la aplicación no puede funcionar de forma segura, tales como las credenciales de acceso a la base de datos principal, variables de entorno globales o librerías de autenticación de usuarios.
- **require_once** : Garantiza la inclusión de un archivo esencial una única vez en todo el ciclo de vida de la solicitud; si el archivo ya fue cargado, ignora la nueva orden, pero si el archivo no existe en el disco, detiene la ejecución del programa emitiendo un error fatal. Se utiliza principalmente en el punto de entrada de aplicaciones para declarar clases de la programación orientada a objetos, gestores de dependencias estructurales o

configuraciones de enrutadores de enlaces, donde una doble carga rompería la lógica del compilador.

19. Superglobales

- **\$_GET** : Recopila y almacena todas las variables enviadas al script actual a través de los parámetros de la URL (Query String), estos datos se transmiten de forma visible en la barra de direcciones del navegador después del símbolo ?, decodificando automáticamente estos pares de clave y valor y los pone a disposición del desarrollador en esta estructura global.
Se utiliza para la navegación entre páginas, la implementación de filtros de búsqueda, la gestión de paginación y el paso de identificadores públicos que no requieren seguridad crítica, permitiendo que el estado de una consulta sea compartible mediante un enlace directo.
- **\$_POST** : Contiene los datos enviados al script mediante el método de transmisión HTTP POST, diferencia de otros métodos la información viaja dentro del cuerpo de la petición HTTP, lo que impide que los datos sean visibles en la URL y permite el envío de volúmenes de información mucho mayores, como archivos o textos extensos, procesando el contenido del cuerpo de la solicitud (normalmente con el tipo de contenido application/x-www-form-urlencoded o multipart/form-data) y organiza los campos del formulario en esta variable.
Se utiliza para el envío de formularios de registro, procesamiento de credenciales de acceso, creación de contenido en bases de datos y cualquier operación que implique una modificación del estado del servidor.
- **\$_SERVER** : Contiene información detallada sobre el entorno de ejecución del script, el servidor web y la conexión del cliente, sus índices son generados por el servidor web (como Apache o Nginx) y proporcionan datos críticos como rutas de archivos, cabeceras de la petición, direcciones IP del visitante, el nombre del host y el método de solicitud empleado.
Se utiliza para determinar la ubicación física del script en el servidor, validar el agente de usuario del navegador, realizar redirecciones dinámicas basadas en el dominio y gestionar la lógica de seguridad mediante la detección de protocolos HTTPS o direcciones de origen.
- **\$_REQUEST** : Funciona como un contenedor unificado en PHP, agrupando los datos provenientes de las variables externas de _GET, _POST y _COOKIE, el motor de PHP inicializa esta estructura al comienzo de cada petición, permitiendo al desarrollador acceder a los parámetros enviados por el cliente sin necesidad de especificar el método de transmisión original a través del cual llegaron los datos, el orden de prioridad en caso de que existan claves duplicadas entre las diferentes fuentes está definido por la directiva request_order en la configuración del servidor.
Se utiliza en scripts que requieren una recepción de datos flexible, facilitando la captura de identificadores o variables de control que pueden ser enviadas indistintamente mediante formularios, parámetros de URL o persistencia de navegador.

20. Manejo de sesiones

- **session_start()** : Instrucción que inicia una nueva sesión o reanuda una existente basada en un identificador de sesión enviado a través de una cookie o un parámetro GET, al ejecutarse el motor de PHP busca un archivo de sesión correspondiente en el almacenamiento temporal del servidor, si lo encuentra recupera los datos previamente guardados y si no, genera un nuevo identificador único (PHPSESSID). Esta función debe ser invocada antes de que el script envíe cualquier tipo de salida HTML o cabecera HTTP al navegador, ya que requiere la modificación de las cabeceras de respuesta para establecer la cookie de sesión y se utiliza para habilitar el seguimiento del usuario a través de múltiples páginas, permitiendo que el servidor reconozca al cliente en cada petición sucesiva.
- **\$_SESSION** : Almacena datos persistentes en el servidor asociados a un usuario específico durante su visita al sitio web, a diferencia de otras estructuras de datos que se destruyen al finalizar la ejecución del script, los valores guardados en este array permanecen disponibles para futuras peticiones del mismo usuario mientras la sesión no expire o sea destruida manualmente, PHP serializa automáticamente el contenido de este array y lo guarda en el sistema de archivos o en memoria del servidor al terminar el script.
Se utiliza para mantener el estado de autenticación de un usuario, gestionar carritos de compras en sitios de comercio electrónico, almacenar preferencias temporales y transferir mensajes informativos (flash messages) entre diferentes páginas de una aplicación.

21. Cookies

- **setcookie()** : Mecanismo definido en el núcleo de PHP para enviar una cookie HTTP desde el servidor hacia el navegador del cliente, al invocarse PHP genera una cabecera Set-Cookie que el navegador interpreta y almacena en el disco local o en la memoria volátil según los parámetros especificados, esta función permite definir no solo el nombre y el valor del dato, sino también su tiempo de expiración (Unix timestamp), la ruta del servidor donde será válida, el dominio, y banderas de seguridad como httponly (para evitar acceso mediante JavaScript) o secure (para transmisión exclusiva vía HTTPS).
Se utiliza para implementar funciones de "recordar sesión", rastrear preferencias estéticas del usuario, gestionar carritos de compra persistentes y almacenar identificadores de análisis de marketing sin intervención directa del servidor en cada carga.
- **\$_COOKIE** : Contiene todos los datos que el navegador del cliente envía de vuelta al servidor dentro de las cabeceras de la petición HTTP, en el que el motor de PHP decodifica automáticamente estas cabeceras al inicio del script y organiza los pares clave-valor dentro de esta estructura global para que estén disponibles inmediatamente, es importante notar que los datos contenidos en este array corresponden a los valores que el navegador ya tenía almacenados antes de que el script actual comenzara su ejecución, por lo tanto un cambio realizado con setcookie() no será visible en esta variable hasta la siguiente recarga de la página. Se utiliza para validar estados de autenticación persistentes, recuperar

configuraciones de personalización local y leer identificadores de seguimiento que permiten al servidor reconocer a un cliente recurrente de forma única.

22. Manejo de headers

- **header()** : Envía una cabecera HTTP sin procesar directamente al cliente (navegador), las cabeceras son metadatos que viajan antes del contenido real del documento y definen cómo debe interpretarse la respuesta del servidor, es un requisito crítico que esta función sea invocada antes de que el script genere cualquier tipo de salida de texto, etiquetas HTML o espacios en blanco, ya que una vez que el cuerpo de la respuesta comienza a enviarse, las cabeceras quedan bloqueadas y cualquier llamada posterior resultará en un error de tipo "headers already sent".

Se utiliza para definir el tipo de contenido (MIME type), configurar políticas de caché, establecer parámetros de seguridad como Content-Security-Policy y gestionar la descarga forzada de archivos mediante la disposición de contenido.

- **Redirecciones** : Implementación específica de la función header() que utiliza el campo Location para indicar al navegador que debe solicitar una URL diferente de forma automática, que cuando el servidor envía una cabecera de este tipo (generalmente acompañada de un código de estado HTTP 301 para redirecciones permanentes o 302 para temporales), el navegador interrumpe la carga actual y realiza una nueva petición hacia la dirección especificada.
Es una práctica estándar de seguridad y lógica invocar la función exit() o die() inmediatamente después de una instrucción de redirección para asegurar que el motor de PHP detenga el procesamiento del script original y no ejecute código residual que podría ser vulnerable o innecesario y se utiliza para redirigir usuarios tras un inicio de sesión exitoso, enviar a los visitantes a páginas de error controladas o gestionar el flujo de navegación tras procesar formularios de envío de datos.

23. Funciones comunes

- **Strings**

|
| — **explode()** : Segmenta una cadena de texto en un array de subcadenas, utilizando un delimitador específico para marcar los puntos de ruptura, el motor de PHP recorre la cadena de origen y, cada vez que encuentra el separador definido (que puede ser un solo carácter o una secuencia), extrae el fragmento de texto anterior y lo almacena como un nuevo elemento en la colección resultante, si el delimitador es una cadena vacía, la función devolverá un error; si el delimitador no se encuentra en el texto, retornará un array con un único elemento que contiene la cadena original íntegra.

| Se utiliza para procesar datos estructurados en formato CSV, descomponer rutas de directorios, extraer etiquetas separadas por comas y convertir oraciones en listas de palabras individuales para su análisis lógico.

|
| — **implode()** : Realiza la operación inversa a la segmentación, uniendo todos los elementos de un array en una única cadena de texto mediante un nexo o pegamento (glue) definido por el desarrollador, durante este proceso PHP

concatena secuencialmente cada valor de la colección e inserta el delimitador elegido entre ellos, omitiéndolo después del último elemento para garantizar una estructura de texto limpia, siendo capaz de manejar arrays indexados y asociativos, aunque solo utiliza los valores para la construcción de la cadena final.

Se utiliza habitualmente para generar consultas SQL dinámicas (como en la cláusula IN), construir listas de destinatarios de correo separados por puntos y coma, y transformar colecciones de datos procesados en formatos legibles para la capa de presentación o exportación.

- **Arrays**

- |
 - **in_array()** : Comprueba si un valor específico existe dentro de los elementos de un array, al ejecutarse el motor de PHP realiza un recorrido secuencial sobre la colección y compara el valor buscado con cada uno de los valores almacenados y si encuentra una coincidencia, devuelve true, de lo contrario devuelve false.
Por defecto, esta función realiza una comparación débil (==), pero admite un tercer parámetro booleano opcional que, al activarse, fuerza una comparación estricta (===) de valor y tipo de dato y se utiliza para validar si una opción seleccionada por el usuario pertenece a una lista de opciones permitidas, verificar la existencia de permisos en un sistema de roles y controlar el flujo lógico basado en la presencia de datos dentro de una lista indexada.
 - **array_key_exists()** : Verifica si una clave o índice específico ha sido definido dentro de un array, a diferencia de otras funciones de validación, esta consulta directamente la estructura de punteros del array y devuelve true incluso si el valor asociado a esa clave es null, siendo una operación de búsqueda de alta eficiencia que se centra exclusivamente en la existencia del identificador (ya sea un entero o un string) y no en el contenido que este protege.
Se utiliza para prevenir errores de tipo "undefined index" al intentar acceder a posiciones de un array, validar la presencia de configuraciones obligatorias en diccionarios de datos y asegurar que una clave de un formulario o API ha sido enviada antes de intentar procesar su valor.

- **Formato**

- |
 - **number_format()** : Convierte un número de punto flotante o entero en una cadena de texto formateada con separadores de miles y decimales, el motor de PHP procesa el valor numérico y le aplica reglas de redondeo matemático según la cantidad de decimales especificada por el desarrollador.
Esta función permite una configuración personalizada mediante cuatro parámetros: el número a convertir, la cantidad de decimales, el carácter para el punto decimal y el carácter para el separador de miles y se utiliza fundamentalmente en la capa de presentación para mostrar montos monetarios, medidas técnicas o estadísticas de forma legible para el usuario final, garantizando que los datos numéricos crudos se transformen en información visualmente estandarizada según las convenciones regionales.

- **Debug**

- └─ **var_dump()** : Muestra información estructurada y detallada sobre una o más expresiones, que a diferencia de las instrucciones de salida simple, esta función revela no solo el valor del dato, sino también su tipo (string, int, float, etc.), su extensión (longitud de cadena o número de elementos en un array) y en el caso de los objetos, su clase y propiedades internas, si se utiliza con estructuras anidadas, como arrays multidimensionales u objetos complejos, la función genera una representación jerárquica que permite visualizar toda la profundidad de los datos.

Se utiliza exclusivamente en entornos de desarrollo y pruebas para depurar errores lógicos, verificar el estado de las variables en puntos críticos de la ejecución y asegurar que las funciones están retornando las estructuras de datos esperadas antes de pasar a producción.

- **Seguridad básica**

- └─ **htmlspecialchars()** : Convierte caracteres especiales en entidades HTML para prevenir que el navegador interprete fragmentos de texto como código ejecutable, el motor de PHP rastrea la cadena de origen y sustituye símbolos reservados (como < por <, > por >, & por & y las comillas) por sus equivalentes de texto plano.

Esta función admite parámetros adicionales para definir el tipo de comillas a convertir (entidades simples o dobles) y la codificación de caracteres (usualmente UTF-8) y se utiliza sistemáticamente como la defensa principal contra ataques de Cross-Site Scripting (XSS), asegurando que cualquier entrada proveniente de un usuario, al ser mostrada de nuevo en la interfaz, se renderice únicamente como texto visual y no como un script malicioso o una alteración de la estructura del DOM.

24. Función de String

- **strlen()** : Devuelve la longitud de una cadena de texto medida en bytes, en el motor de PHP, cada carácter se contabiliza como una unidad de memoria de 8 bits por lo tanto, la función retorna un número entero que representa el tamaño total de la secuencia de caracteres proporcionada.

Su ejecución es de alta velocidad ya que no procesa el contenido de la cadena, sino que consulta directamente el metadato de longitud almacenado en la estructura interna de la variable y se utiliza para validar extensiones máximas de texto, controlar límites de almacenamiento en bases de datos y verificar que los campos de entrada no excedan la capacidad de memoria asignada para su procesamiento.

- **strpos()** : Encuentra la posición numérica de la primera aparición de una subcadena dentro de otra cadena de texto y el valor devuelto es un número entero que indica el índice (comenzando desde 0) donde se localiza el primer carácter de la coincidencia, si la subcadena no se encuentra, la función devuelve el valor booleano false. Esta función realiza una búsqueda sensible a mayúsculas y minúsculas y permite un tercer parámetro opcional para indicar desde qué posición iniciar el escaneo y se

utiliza para detectar la presencia de palabras clave, validar formatos de correos electrónicos o protocolos de URL, y segmentar textos basándose en delimitadores específicos.

25. Validación básica

- **isset()** : Determina si una variable ha sido declarada y si su valor actual es distinto de null, que al ejecutarse el motor de PHP realiza una comprobación rápida en la tabla de símbolos del script y si la variable existe y contiene cualquier valor (incluyendo una cadena vacía "", el número 0 o el booleano false), la función devuelve true, una característica crítica de esta función es que no genera avisos de error (Notice) si la variable que se intenta comprobar no ha sido definida previamente.
Se utiliza sistemáticamente para verificar la presencia de índices en arreglos superglobales como `_GET` o `_POST`, asegurar que las propiedades de un objeto han sido inicializadas y controlar el flujo de ejecución basándose en la existencia física de los datos en memoria.
- **empty()** : Determina si una variable se considera "vacía" desde una perspectiva lógica, una variable se evalúa como vacía si no existe o si su valor es equivalente a false en una conversión booleana automática, esto incluyendo el valor null, el entero 0, el flotante 0.0, una cadena vacía "", la cadena "0", un array vacío [] y variables declaradas pero sin valor asignado, además esta función no emite errores si la variable no está definida.
Se utiliza para validar formularios de usuario donde se requiere que un campo contenga información sustancial, limpiar colecciones de datos de elementos nulos o irrelevantes y simplificar condiciones lógicas donde la ausencia de valor o un valor "cero" deben recibir el mismo tratamiento operativo.
- **is_numeric()** : Comprueba si una variable es un número o una cadena numérica que puede ser interpretada como tal por el motor de PHP, a diferencia de las validaciones de tipo estricto, esta función devuelve true tanto para variables de tipo int y float como para strings que contienen representaciones válidas de números (incluyendo signos, decimales y notación científica).
No válida únicamente la naturaleza del dato, sino su capacidad de ser utilizado de forma segura en operaciones aritméticas y se utiliza de manera extensiva para sanear entradas de usuario antes de realizar cálculos matemáticos, validar parámetros de identificación en URLs y asegurar la integridad de los datos antes de realizar inserciones en campos numéricos de una base de datos.
- **Patrón null coalescing** : Valida la existencia de un valor y asignar un sustituto en una sola operación atómica, su lógica interna evalúa si el operando de la izquierda existe y no es null, si ambas condiciones se cumplen, devuelve dicho valor, de lo contrario devuelve el operando de la derecha, este patrón combina intrínsecamente la seguridad de una comprobación de existencia con la capacidad de definición por defecto, eliminando la necesidad de estructuras condicionales verbosas.
Se utiliza para la inicialización segura de variables a partir de fuentes externas inciertas, la gestión de configuraciones opcionales y la prevención de errores de

ejecución al intentar acceder a claves de arreglos que podrían no haber sido enviadas por el cliente.

26. Manejo de archivos

- **file_exists()** : Verifica si un archivo o directorio específico existe en la ruta indicada del servidor, al ejecutarse el motor de PHP realiza una consulta al sistema de archivos del sistema operativo y devuelve un valor booleano: true si el recurso es localizado y el proceso de PHP tiene permisos de lectura sobre su estructura de directorios, o false en caso contrario, algo importante es notar que los resultados de esta función son almacenados en una caché interna de estado de archivos para optimizar el rendimiento, la cual puede requerir una limpieza manual con `clearstatcache()`.
Se utiliza sistemáticamente como medida de seguridad previa antes de intentar leer, escribir o incluir archivos, evitando errores fatales de ejecución y permitiendo la gestión controlada de recursos faltantes.
- **file_get_contents()** : Mecanismo estándar y más eficiente para leer el contenido íntegro de un archivo y almacenarlo en una cadena de texto (string), en que la función mapea el archivo en la memoria del servidor en una sola operación atómica, lo que la hace ideal para procesar archivos de tamaño moderado, además de archivos locales, si la configuración del servidor lo permite, puede recuperar el contenido de recursos remotos a través de protocolos como HTTP o FTP.
Se utiliza habitualmente para cargar archivos de configuración JSON o XML, leer plantillas HTML externas, capturar respuestas de APIs de terceros y procesar registros de log textuales para su análisis o visualización en la interfaz del administrador.
- **file_put_contents()** : Escribe una cadena de datos dentro de un archivo específico, simplificando el proceso de escritura al realizar internamente las tareas de apertura, escritura y cierre del puntero de archivo en un solo paso, por defecto si el archivo ya existe, su contenido previo es sobrescrito, sin embargo, admite el uso de la bandera "FILE_APPEND" para añadir los nuevos datos al final del archivo existente sin borrar lo anterior, también puede crear el archivo automáticamente si este no existe, siempre que el proceso de PHP cuente con los permisos de escritura necesarios en el directorio destino.
Se utiliza de forma extensiva para generar archivos de registro (logs), almacenar respuestas de caché estáticas, guardar configuraciones dinámicas y exportar datos procesados a formatos de texto plano.

27. Tiempo

- **date()** : Transforma un timestamp de Unix en una cadena de texto formateada según las preferencias del desarrollador, requiriendo un string de formato que utiliza caracteres reservados (como 'Y' para el año, 'm' para el mes o 'H' para la hora) para construir la representación visual de la fecha y hora, si no se proporciona un segundo parámetro con un timestamp específico, la función utiliza automáticamente el valor devuelto por el servidor en el instante de la ejecución y el resultado final se ve afectado por la configuración de la zona horaria predeterminada del script (`date.default_timezone_set`).

Se utiliza para mostrar fechas legibles en interfaces de usuario, generar nombres de archivos basados en el tiempo, estructurar reportes cronológicos y validar formatos de fecha en procesos de exportación de datos.

- **time()** : Devuelve el momento actual medido como el número de segundos transcurridos desde la Época Unix (1 de enero de 1970 00:00:00 GMT), este valor es un número entero (integer) conocido como timestamp, que representa una medida de tiempo absoluta e independiente de la zona horaria o del formato de visualización.

Al ser una unidad aritmética simple, es extremadamente eficiente para realizar cálculos temporales, como determinar la diferencia de días entre dos fechas o establecer tiempos de expiración para cookies y sesiones y se utiliza como la fuente de datos base para registrar momentos de creación en bases de datos, comparar instantes de ejecución y alimentar funciones de formateo que requieren un punto de referencia cronológico preciso.

28. JSON

- **json_encode()** : Convierte una estructura de datos de PHP (como un array o un object) en una cadena de texto con formato JSON (JavaScript Object Notation), durante este proceso de serialización, el motor de PHP traduce los tipos de datos nativos a sus equivalentes en el estándar JSON: los arreglos indexados se transforman en arrays de corchetes [], los arreglos asociativos en objetos de llaves {} y los valores escalares mantienen su naturaleza (strings, números o booleanos). Esta función admite una serie de constantes de configuración (como JSON_PRETTY_PRINT para legibilidad o JSON_UNESCAPED_UNICODE para caracteres especiales) que modifican el resultado final de la cadena y se utiliza de forma sistemática para enviar respuestas de datos desde el servidor hacia aplicaciones de front-end, generar archivos de configuración intercambiables y almacenar estructuras de datos complejas en bases de datos documentales.
- **json_decode()** : Transforma una cadena de texto en formato JSON en una estructura de datos nativa de PHP, y por defecto esta función convierte los objetos JSON en instancias de la clase genérica stdClass, sin embargo permite un segundo parámetro booleano que, al ser establecido como true, fuerza la conversión de toda la estructura en arrays asociativos anidados.
Es una función crítica para la comunicación externa, ya que procesa la información recibida desde peticiones de clientes, integraciones con servicios de terceros o lectura de archivos de configuración externos y se utiliza para interpretar cuerpos de solicitudes HTTP en servicios RESTful, decodificar respuestas de APIs externas y transformar datos persistidos en texto plano hacia variables lógicas que el script puede manipular aritméticamente o mediante estructuras de control.

29. Manejo de errores básico

- **Error_reporting()** : Establece en tiempo de ejecución qué niveles de error de PHP deben ser capturados y procesados por el motor, clasificando los incidentes en diversas categorías, como errores fatales (E_ERROR), advertencias (E_WARNING), avisos de degradación (E_DEPRECATED) o simples notificaciones de estilo (E_NOTICE).

Esta función permite al desarrollador filtrar estas categorías mediante constantes predefinidas y operadores bit a bit (por ejemplo, `E_ALL & ~E_NOTICE` para reportar todo excepto avisos menores) y se utiliza para personalizar la sensibilidad del motor de PHP según el entorno, permitiendo un nivel de detalle máximo durante la etapa de desarrollo y restringiendo la captura a errores críticos en entornos de producción para evitar la sobrecarga de logs.

- | — **display_errors** : Directiva de configuración (ajustable mediante `ini_set()`) que determina si los errores capturados por el motor deben imprimirse directamente en la salida de pantalla (HTML) o mantenerse ocultos al usuario final, esta configuración no afecta si el error ocurre o no, sino únicamente su representación visual en el navegador.
| En entornos de desarrollo, se establece en On para facilitar la depuración inmediata del código, en entornos de producción, es una norma crítica de seguridad establecerla en Off y se utiliza para prevenir la exposición de información sensible del servidor, como rutas de archivos, nombres de variables o estructuras de consultas SQL, que podrían ser utilizadas por atacantes para identificar vulnerabilidades en la infraestructura.
- | — **set_error_handler()** : Permite al desarrollador definir una función de retorno (callback) personalizada para gestionar los errores generados por el motor de PHP, sustituyendo el comportamiento por defecto del lenguaje, cuando ocurre un error no fatal, el flujo de ejecución se desvía hacia esta función definida, la cual recibe parámetros detallados como el nivel del error, el mensaje descriptivo, el archivo y la línea donde se originó.
| Se utiliza para implementar sistemas de log personalizados que envían alertas por correo electrónico, registrar incidentes en bases de datos externas, o transformar errores tradicionales de PHP en una interfaz visual amigable y coherente con el diseño de la aplicación.
- | — **set_exception_handler()** : Establece una función de gestión para todas aquellas excepciones que no han sido capturadas por un bloque try/catch dentro del código, una vez que se invoca este manejador, el script detiene su ejecución normal, pero permite que la función definida realice tareas finales de limpieza o registro antes de finalizar completamente.
| A diferencia del manejador de errores, este se ocupa específicamente de objetos de la clase Exception o Error (en PHP 7+) y se utiliza como la última línea de defensa en aplicaciones robustas para mostrar páginas de "Error 500" personalizadas, asegurar el cierre de conexiones críticas y garantizar que ningún fallo inesperado termine en una pantalla en blanco o con fugas de información técnica hacia el cliente.